

Funcam

Passi davanti alla webcam e ti vedi sul pannello, con pochi colori

1. L’idea

Una webcam USB attaccata al Raspberry, e sul DMD compare quello che vede — ridotto a quello che un computer di quarant’anni fa sapeva mostrare.

Non è un filtro nostalgico appiccicato sopra. È quello che questo pannello sa fare davvero.

2. Perché si parte da otto colori

Sta scritto da tempo nel Now Playing, sotto `safe_colors`: **su questo pannello gli otto colori pieni sono gli unici che non sfarfallano**. Ogni componente a 0 o a 255, niente vie di mezzo. Era nato come rimedio a un difetto — le tinte intermedie tremano — e si è scoperto che è esattamente la tavolozza dei primi computer a colori.

Il vincolo hardware e l’estetica voluta sono la stessa cosa. Capita di rado, e qui si sfrutta invece di combatterlo.

Le sfumature che mancano le rimette il **dithering ordinato** di Bayer: una matrice 4×4 di soglie che alterna i pixel accesi e spenti secondo un motivo regolare. Da vicino si vede la trama, da lontano si vedono i mezzitoni. È come stampavano i giornali, ed è come disegnavano i computer a otto colori.

Nella pagina **Funcam** si sceglie fra tre aspetti:

Aspetto	Cosa fa
Colori	Il cubo dei colori, con dithering. Quanti sono lo decidi tu: vedi il capitolo 3.
Verde Game Boy	Le quattro tinte dello schermo DMG, le stesse della sorgente Game Boy.
Grigi	Da 2 a 16 livelli di grigio. Regge bene la poca luce della sera.

Quanti colori, davvero

La regola degli otto colori è nata disegnando **testo**: lettere chiare su fondo nero, dove i pochi pixel sfumati del bordo tremano contro uno sfondo fermo. È la condizione in cui il difetto si vede peggio.

Un’immagine di telecamera è un’altra cosa. È fatta *tutta* di mezzi toni, non c’è nessun bordo netto a cui confrontarli, e l’occhio legge il tutto come grana. Non è detto che tremi allo stesso modo — e non è una cosa che si possa decidere ragionando.

Per questo i livelli si scelgono dalla pagina, da 2 a 8 **per canale**. I colori sono il loro cubo:

Livelli	Colori	
2	8	i pieni, quelli sicuri — il predefinito
3	27	
4	64	
6	216	in pratica la tavolozza da 256 colori dell’epoca
8	512	

Un 256 esatto non esiste con tre canali uniformi: i 256 colori dei computer di allora erano una tavolozza scelta a mano, non tre canali indipendenti. 216 è il numero vicino più onesto, ed è anche la vecchia *web safe palette*, nata dallo stesso conto.

Salire **non costa niente**: gli stessi byte, lo stesso conto vettoriale, nessun traffico in più sul bus. Costa solo il rischio che le tinte intermedie sfarfallino, e l'unico modo di saperlo è guardare il pannello. Con più livelli, in compenso, il dithering ha passi più corti da coprire: la trama diventa più fine e l'immagine meno granulosa.

Il valore predefinito resta 2, perché un predefinito deve funzionare senza chiedere niente a nessuno.

Il contrasto automatico

Un soggiorno di sera, dalla webcam, esce come una poltiglia grigia stretta fra 40 e 90. Ridotta a pochi colori diventerebbe un rettangolo quasi uniforme.

Prima di quantizzare si allarga la gamma usando il 2% e il 98% dei valori: è ciò che fa la differenza fra vedere qualcosa e vedere una macchia. Se però la scena è *davvero* piatta — una parete, il buio totale — allargare amplificherebbe solo il rumore del sensore fino a farne neve, quindi sotto una certa differenza si lascia stare.

3. Il carico, che su questo pannello si vede

Una telecamera che macina fotogrammi è precisamente il tipo di lavoro che produce le righe chiare: la campagna di misure aveva mostrato **0,95%** di fotogrammi disturbati a riposo contro **8,90%** con la scheda SD sotto sforzo. Traffico sul bus di memoria si paga in righe luminose sul pannello.

Tre difese, in ordine di efficacia.

Non chiedere i fotogrammi che non servono

Questa è la più importante, e la differenza è sottile ma reale.

Scartare un fotogramma dopo la cattura risparmia solo CPU: quel fotogramma ha già attraversato l'USB e il bus di memoria. **Non chiederlo** risparmia tutto, perché non esiste mai.

Quindi il numero di fotogrammi al secondo non è un filtro a valle: è `-framerate` passato all'ingresso v4l2, cioè quello che si chiede alla telecamera di **produrre**. Dieci al secondo bastano; sotto i cinque l'immagine va a scatti.

Non decodificare

Si chiede **YUYV** e non MJPEG. Nessuna decodifica JPEG da pagare: i pixel arrivano già pronti. Il ritaglio e il ridimensionamento li fa ffmpeg, che è compilato per farlo, e a Python arrivano direttamente i 256×64 finali.

La riduzione usa `flags=area` e non `neighbor`: da 640 a 256, il vicino più prossimo produce alias e bordi che sfarfallano. La grana da otto bit la mette il dithering, non un ridimensionamento fatto male.

Spegnere quando nessuno guarda

Se il pannello è occupato da ZeDMD o da una partita a Doom, la telecamera non serve a nessuno. Dopo venti secondi che nessuno chiede fotogrammi la cattura si ferma da sola, e riparte alla prima richiesta. È il risparmio più grosso, perché è totale.

Nel secondo che ffmpeg impiega a tornare si continua a mostrare l'ultimo fotogramma, per non lasciare un buco nero. Ma solo per dieci secondi: se la webcam è stata staccata, un'immagine congelata che resta lì per sempre è peggio del nero, perché sembra che funzioni.

4. Dove sta, e chi ha la precedenza

Funcam ha **priorità 51**: sopra il Media Player (50), sotto il Rolling Banner (55).

Acceso il servizio, la ripresa dal vivo è quello che si vuole vedere, quindi sta sopra la rotazione di foto e video, che altrimenti la interromperebbe a intervalli casuali. Ma resta sotto tutto ciò che ha qualcosa da **dire**: scritte, compleanni, scadenze, calendario, musica, aerei, e naturalmente ZeDMD. Un avviso che non compare perché c'è la telecamera accesa sarebbe un avviso perso.

51 e non 50: a parità l'arbitro tiene chi ha registrato per primo, e un pareggio vorrebbe dire una sorgente che non compare mai. Una prova lo verifica per tutte le sorgenti insieme.

5. Scegliere la telecamera

Una webcam USB non espone **un** `/dev/video`: ne espone due o più. Il primo cattura immagini, gli altri portano metadati e non danno un fotogramma nemmeno a insistere. Un elenco che li mostrasse tutti proporrebbe due voci con lo stesso nome, di cui una non funziona, e chi sceglie non avrebbe modo di sapere quale.

Si raggruppa quindi per dispositivo fisico e si tiene il nodo di numero più basso, che per le webcam UVC è quello di cattura. È una regola pratica, non una garanzia: se la scelta non funziona, lo stato in cima alla pagina lo dice.

La risoluzione di cattura va scelta fra quelle che la telecamera sa dare davvero:

```
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats-ext
```

Più grande non vuol dire meglio: l'immagine finisce comunque in 256×64, e ogni pixel in più è traffico pagato per niente.

6. Foto e mini video

Due pulsanti. Il primo salva il fotogramma attuale in PNG, il secondo registra qualche secondo in GIF animata.

Finiscono nella **libreria media**, sotto la sottocartella `telecamera`. Non è un dettaglio organizzativo: significa che il Media Player le rimette sul pannello da solo, più avanti e senza che nessuno glielo chieda. Uno scatto di stasera che ricompare fra una settimana è metà del divertimento.

La GIF si salva in un thread a parte, perché scrivere un file mentre il pannello disegna è proprio il carico che genera le righe. Con otto colori la tavolozza è minuscola e il file pesa quanto un'icona.

7. Dove finiscono le immagini

Da nessuna parte.

Le immagini non escono dal Raspberry: niente rete, niente MQTT, nessun servizio esterno. Quello che salvi resta sulla scheda SD, e si cancella dalla libreria media come qualunque altro contenuto.

Il servizio parte **spento**, e non è prudenza formale: è una telecamera accesa in soggiorno. Si accende quando lo decidi tu, dalla pagina Servizi — o da Home Assistant, dove c'è il suo interruttore. Che una telecamera si possa spegnere anche da lontano è la ragione principale per cui quell'interruttore vale la pena averlo.